

Как мы уже видели, контейнеризовать можно почти любой проект. Поскольку мы находимся между Dev и Ops, давайте представим, что некоторые наши коллеги-разработчики сделали приложение с README, который инструктирует, что устанавливать и как запускать приложение. Теперь мы, как эксперты по контейнерам, можем контейнеризовать его за секунды.

Откройте этот проект <https://github.com/docker-hy/material-applications/tree/main/rails-example-project> и прочитайте README и подумайте, как превратить его в Dockerfile. Благодаря README мы должны быть в состоянии расшифровать, что нам нужно сделать, даже если мы не имеем понятия о языке или технологии!

Нам нужно клонировать [репозиторий](#), который вы, возможно, уже сделали. После этого давайте начнем с Dockerfile. Мы знаем, что нам нужно установить Ruby и какие бы зависимости ни были у него. Давайте поместим Dockerfile в корень проекта.

Dockerfile

```
# We need ruby 3.1.0. I found this from Docker Hub
FROM ruby:3.1.0

EXPOSE 3000

WORKDIR /usr/src/app
```

Хорошо, это основы, у нас есть FROM с версией Ruby, EXPOSE 3000 было указано внизу README и WORKDIR /usr/src/app — это соглашение.

Следующее нам сообщается README. Нам не нужно копировать ничего извне контейнера, чтобы запустить это:

```
# Install the correct bundler version
RUN gem install bundler:2.3.3

# Copy the files required for dependencies to be installed
COPY Gemfile* ./

# Install all dependencies
RUN bundle install
```

Здесь мы использовали быструю уловку, чтобы отделить установку зависимостей от части, где мы копируем исходный код. COPY скопирует оба файла Gemfile и Gemfile.lock в текущую директорию. Это поможет нам, кэшируя слои зависимостей, если нам когда-либо понадобится внести изменения в исходный код. Тот же вид уловки кэширования работает во многих других языках или фреймворках, таких как Node.js.

И наконец, мы копируем проект и следуем инструкциям в README:

```
# Copy all of the source code
COPY . .

# We pick the production mode since we have no intention of developing the software
inside the container.
# Run database migrations by following instructions from README
```

```
RUN rails db:migrate RAILS_ENV=production

# Precompile assets by following instructions from README
RUN rake assets:precompile

# And finally the command to run the application
CMD ["rails", "s", "-e", "production"]
```

Хорошо. Давайте посмотрим, насколько хорошо сработало копирование README, и запустим следующий one-liner, который собирает образ, а затем запускает его с опубликованным портом 3000:

```
docker build . -t rails-project && docker run -p 3000:3000 rails-project
```

После некоторого ожидания приложение запускается на порту 3000 в production режиме... если у вас нет Mac с процессором M1 или M2.

:::tip Сборка образа с более новым Mac

Если у вас более новый Mac с процессором [M1 или M2](#), сборка образа завершается ошибкой:

```
=> ERROR [7/8] RUN rails db:migrate RAILS_ENV=production
-----
> [7/8] RUN rails db:migrate RAILS_ENV=production:
#11 1.142 rails aborted!
#11 1.142 LoadError: cannot load such file -- nokogiri
```

Это можно исправить, изменив следующую строку в файле *Gemfile.lock*

```
nokogiri (1.13.1-x86_64-darwin)
```

на форму:

```
nokogiri (1.14.2-arm64-darwin)
```

Причина проблемы в том, что файл *Gemfile.lock*, который определяет *точные* версии установленных библиотек (или Gems в терминологии Ruby), генерируется с Linux, имеющим процессор Intel. Gem [Nokogiri](#) имеет разные версии для процессоров Intel и Apple M1/M2, и чтобы получить правильную версию Gem для более нового Mac, сейчас проще всего сделать изменение в файле *Gemfile.lock*.

:::

Упражнения 1.11-1.14

:::info Упражнение 1.11: Spring

Создайте Dockerfile для старого Java Spring проекта, который можно найти в [репозитории курса](#).

Настройка должна быть простой с инструкциями README. Советы, чтобы начать:

Есть много вариантов запуска Java, вы можете использовать, например, [amazoncorretto](#) FROM `amazoncorretto:_tag_`, чтобы получить Java вместо ручной установки. Выберите тег, используя

README и страницу Docker Hub.

Вы завершите упражнение, когда увидите сообщение 'Success' в вашем браузере.

Отправьте Dockerfile, который вы использовали для запуска контейнера.

...

Следующие три упражнения начнут более крупный проект, который мы будем настраивать в частях 2 и 3. Они потребуют от вас использовать все, что вы узнали до сих пор. Если вам нужно изменить Dockerfile в каких-либо последующих упражнениях, не стесняйтесь делать это поверх Dockerfiles, которые вы создаете здесь.

:::warning Обязательные упражнения Следующие упражнения являются первыми обязательными. Обязательные упражнения нельзя пропускать. :::

:::caution Обязательное Упражнение 1.12: Привет, frontend!

Хороший разработчик создает хорошо написанные README. Такие, что их можно использовать для создания Dockerfiles с легкостью.

Клонируйте, форкните или скачайте проект из <https://github.com/docker-hy/material-applications/tree/main/example-frontend>.

Создайте Dockerfile для проекта (example-frontend) и дайте команду, чтобы проект запускался в Docker контейнере с открытым и опубликованным портом 5000, так что когда вы запустите контейнер и перейдете на <http://localhost:5000> вы увидите сообщение, если вы успешны.

- обратите внимание, что порт 5000 зарезервирован в более новых версиях OSX (Monterey, Big Sur), так что вам придется использовать другой порт хоста

Отправьте Dockerfile.

Как и в других упражнениях, не изменяйте код проекта

СОВЕТЫ:

- В проекте есть инструкции по установке в README.
- Обратите внимание, что приложение начинает принимать соединения, когда "Accepting connections at <http://localhost:5000>" было выведено на экран, это занимает несколько секунд
- Вам не нужно устанавливать ничего нового вне контейнеров.
- Проект может не работать с слишком новыми версиями Node.js

...

:::caution Обязательное Упражнение 1.13: Привет, backend!

Клонируйте, форкните или скачайте проект из <https://github.com/docker-hy/material-applications/tree/main/example-backend>.

Создайте Dockerfile для проекта (example-backend). Запустите контейнер с опубликованным портом 8080.

Когда вы запустите контейнер и перейдете на <http://localhost:8080/ping>, вы должны получить "pong" в ответ.

Отправьте Dockerfile и использованную команду.

Не изменяйте код проекта

СОВЕТЫ:

- вам может понадобиться [это](#)
- Если у вас Mac M1/M2, вам может понадобиться сборка образа с дополнительной опцией `docker build --platform linux/amd64 -t imagename .`

...

Обязательное Упражнение 1.14: Окружение

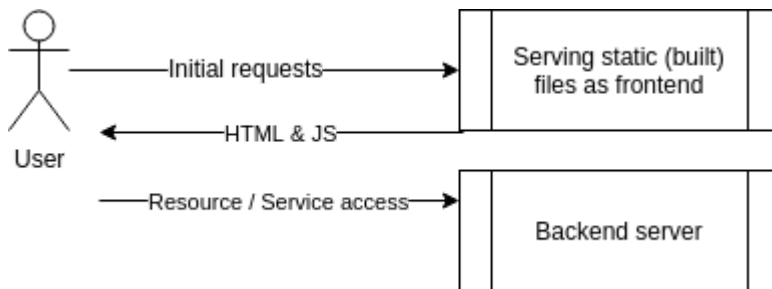
Запустите `frontend`, и `backend` с правильными открытыми портами и добавьте [ENV](#) в `Dockerfile` с необходимой информацией из обоих README ([front](#), [back](#)).

Игнорируйте конфигурации `backend` до тех пор, пока `frontend` не отправит запросы на `_backend_url/ping`, когда вы нажмете кнопку.

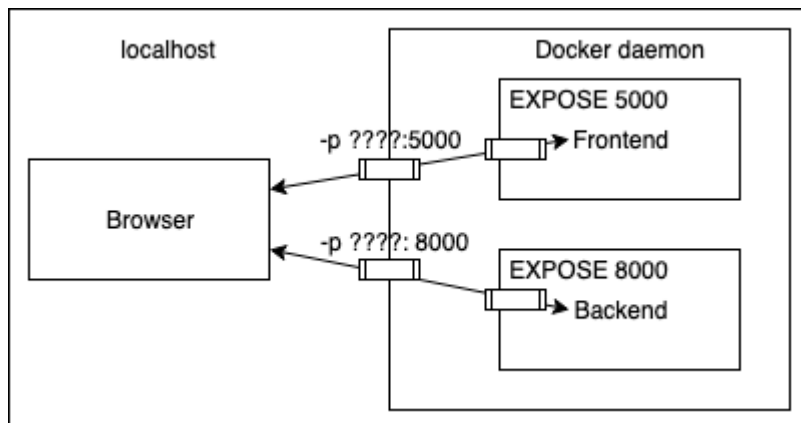
Вы знаете, что конфигурация готова, когда кнопка для 1.14 `frontend` отвечает и становится зеленой.

Не изменяйте код ни одного из проектов

Отправьте отредактированные `Dockerfiles` и использованные команды для запуска.



Frontend сначала будет общаться с вашим браузером. Затем код будет выполняться из вашего браузера, и это отправит сообщение backend.



СОВЕТЫ:

- При настройке веб-приложений держите консоль разработчика браузера ВСЕГДА открытой, F12 или `cmd+shift+I`, когда окно браузера открыто. Информация о настройке междоменных запросов находится в README backend проекта.

- Консоль разработчика имеет несколько видов, самые важные — Console и Network. Изучение вкладки Network может дать вам много информации о том, куда отправляются сообщения и что получается в ответ!

...

Публикация проектов

Перейдите на <https://hub.docker.com/>, чтобы создать аккаунт. Вы можете настроить Docker hub для сборки ваших образов для вас, но использование `push` также работает.

Давайте опубликуем образ youtube-dl. Войдите и перейдите к вашей [панели управления](#) и нажмите Create Repository. Пространство имен может быть либо вашим личным аккаунтом, либо аккаунтом организации. Пока давайте придерживаться личных аккаунтов и напомним что-то описательное, например youtube-dl, в качестве имени репозитория. Нам нужно будет запомнить это в части 2.

Установите видимость на *public*.

И последнее, что нам нужно — это аутентифицировать наш push, войдя в систему:

```
$ docker login
```

Далее вам нужно будет переименовать образ, чтобы включить ваше имя пользователя, а затем вы сможете его запустить:

```
$ docker tag youtube-dl <username>/<repository>
...

$ docker push <username>/<repository>
...
```

Упражнения 1.15-1.16

:::info Упражнение 1.15: Домашнее задание

Создайте Dockerfile для приложения или любого другого dockerized проекта в любом из ваших собственных репозиториях и опубликуйте его на Docker Hub. Это может быть любой проект, кроме клонов или форков backend-example или frontend-example.

Для выполнения этого упражнения вы должны предоставить ссылку на проект в Docker Hub, убедитесь, что у вас есть хотя бы базовое описание и инструкции по запуску приложения в [README](#), доступном через вашу отправку.

...

:::info Упражнение 1.16: Облачное развертывание

Пришло время завершить эту часть и запустить контейнеризованное приложение в облаке.

Вы можете взять любое веб-приложение, например, пример или упражнение из этой части, свое собственное приложение или даже материалы курса (см. [devopsdockeruh/coursepage](#)) и развернуть его в каком-либо облачном провайдере.

Есть много альтернатив, и большинство предоставляют бесплатный уровень. Вот некоторые альтернативы, которые довольно просты в использовании:

- fly.io (прост в использовании, но требует кредитную карту даже в бесплатном уровне)
- render.com (плохая документация, вам, скорее всего, понадобится google)
- heroku.com (имеет бесплатный студенческий план через [GitHub Student Developer Pack](https://github.com/GitHubStudentDeveloperPack))

Если вы знаете хороший облачный сервис для целей этого упражнения, пожалуйста, расскажите нам (да, мы уже знаем об Amazon AWS, Google Cloud и Azure...).

Отправьте Dockerfile, краткое описание того, что вы сделали, и ссылку на работающее приложение.

...